Revisiting Exact-Penalty Linear Programming

A projection-path walker for the regime where Newton slows down

Jeff Kline

17 August 2026

Status. Public prototype, not a peer-reviewed paper. The planned companion HTML page and reproducibility repository do not yet exist, and this document has no stable release or DOI.

1 The gap

Mangasarian showed that a quadratic penalty program yields an exact LP solution at a sufficiently large but finite penalty parameter [Mangasarian and Meyer, 1979, Mangasarian, 1984, 2004]. He used that result to build a Newton method for LPs. Empirically, Newton excels when constraints outnumber variables by roughly 5 to 10 and often reaches the LP solution in only a few nonlinear iterations. But two gaps remain. Newton slows sharply below roughly 2 to 3 constraints per variable, a regime that includes much of the Netlib benchmark panel and arises in practical linear programs. Its stopping tolerances can also compromise numerical accuracy. This note addresses both gaps.

We built a complementary method, the **t-walker**, for this regime. The walker follows the exact dual projection path from face to face instead of solving the penalty problem afresh at a sequence of penalty values. On the tested low-ratio problems, t-walker outpaces Newton. Choosing the faster certified result from t-walker or Newton also improves Mangasarian's performance across Netlib. The pair still trails HiGHS, but it measurably improves on Newton alone.

We test both raw methods against the same original-data accuracy standard, and both pass on the problems they solve. A common simplex crossover cuts their median Netlib error by about three orders of magnitude. After crossover, their median errors have the same order as HiGHS IPM with crossover, although the synthetic tails remain mixed and t-walker still has three Netlib DNFs.

On the current 24-cell synthetic panel (25, 50, and 200 variables; ratios 1.1 through 32; five interleaved runs), the fastest certified walker/Newton result runs a median **2.82 times faster than Newton alone for ratios at most 2**, with cell speedups from 1.69 to 11.76. Across all 24 cells, the measured lower envelope chooses the default-seed walker 11 times, the triangular-seeded walker 6 times, and Newton 7 times. This measured frontier uses hindsight; it does not yet provide an automatic dispatcher.

The standard Netlib-27 panel gives a harder test. The current frontier takes **9.60 s**, versus **11.60 s** for shipped Newton and **0.370 s** for the faster HiGHS engine on each model. Its geometric-mean ratio to HiGHS is **16.6 times**, down from Newton's 26.5 but still far from production parity. That is progress, not victory. HiGHS draws on roughly eighty years of simplex and interior-point ideas and implementation practice [Huangfu and Hall, 2018]; we assembled the walker prototype in a handful of AI-assisted engineering sessions.

2 Two computational strategies

Write the primal and dual as

$$\min_x d^\top x \quad \text{s.t.} \quad Bx \geq b, \qquad \max_y b^\top y \quad \text{s.t.} \quad B^\top y = d,\ y \geq 0,$$

For the dual feasible polyhedron $D = \{y \geq 0 : B^\top y = d\}$, Mangasarian's penalty program returns exactly

$$y(t) = P_D(tb) = \arg\min_{y \in D} \frac{1}{2}\|y - tb\|^2.$$

The optimizer path is continuous and piecewise affine. The optimal penalty value is piecewise quadratic, and the path direction determines its curvature. The walker therefore follows an affine state on each face even though the value function is quadratic there.

Method	What it maintains	Why it wins	Where it pays
Staged Newton	A fixed-t semismooth solve, with staged increases in t and terminal polishing.	Crosses many support changes in a few iterations; excellent on tall synthetic systems.	Expensive identification and factor work near the low-ratio regime.
t-walker	One affine face segment $y(t) = tg + h$ and the next ratio-test event.	Cheap, explicit progress near the square end; checks every accepted endpoint.	Pays for every breakpoint; long paths dominate tall and difficult models.
Triangular-seeded t-walker	The same walk, but a QR/triangular fixed-$t = 0$ projection seed.	Can make initialization and the first face cheaper.	Presently requires full column rank and fails closed on some cells.
HiGHS simplex/IPM	Mature production bases, pricing, presolve, scaling, and crossover.	Robust reference performance and coverage.	A different engineering scale; used as the benchmark, not as our contribution.

3 How the walker advances

On a face with positive dual support W, the projection equations reduce to

$$y_W(t) = tg + h.$$

Each active coordinate $y_i(t)$ must remain nonnegative. Each inactive coordinate has an affine multiplier margin that must also remain nonnegative. The walker finds the next breakpoint with a minimum-ratio test over these margins. At a tie, it exchanges rows at fixed t to resolve degeneracy; those exchanges do not count as progress. A valid next segment must reach a strictly larger t or prove that the path is terminal. Within a face, the penalty curvature is $\phi''(t) = g^\mathsf{T} g$. The terminal test $g^\mathsf{T} g \leq$ tolerance therefore serves as both a geometric stopping rule and the zero-curvature test.

We made the implementation faster and more stable by treating the walker state like a numerical simplex state. The solver now carries the face coefficients, sparse products, support statuses, and a rank-revealing square core across pivots instead of solving unrelated least-squares problems at each one. A one-row change updates that state. The solver fully reconstructs it only after a failed error bound, a rank change, or a rare repair. We made six main changes:

1. native C++ face algebra and cached affine artifacts instead of default library calls at every pivot;
2. a stable square orthogonal core, with a small weak-subspace SVD/COD repair when the face is rank deficient;
3. deterministic statuses for dependent zero rows, so fixed-t exchanges cannot cycle merely by revisiting an equivalent support label;
4. iterative refinement and forward-horizon error bounds that the walker invokes only when coefficient uncertainty can change the next event;
5. fixed-t seed and endpoint repairs that feed the unchanged original-data acceptance gate; and
6. a final primal-dual certificate, independent of t, on the unscaled input.

The prototype remains hybrid. By default, an in-process Newton method builds the walker's $t = 0$ seed. An optional triangular/Wolfe-space seed instead factors B and solves the same fixed-t projection without forming a dense null-space basis. On difficult Netlib faces, the walker can also invoke HiGHS-backed endpoint selection or terminal face repair. We count these calls toward walker time, and every result must pass the original-data certificate. The benchmark therefore does not yet isolate wholly native path algebra.

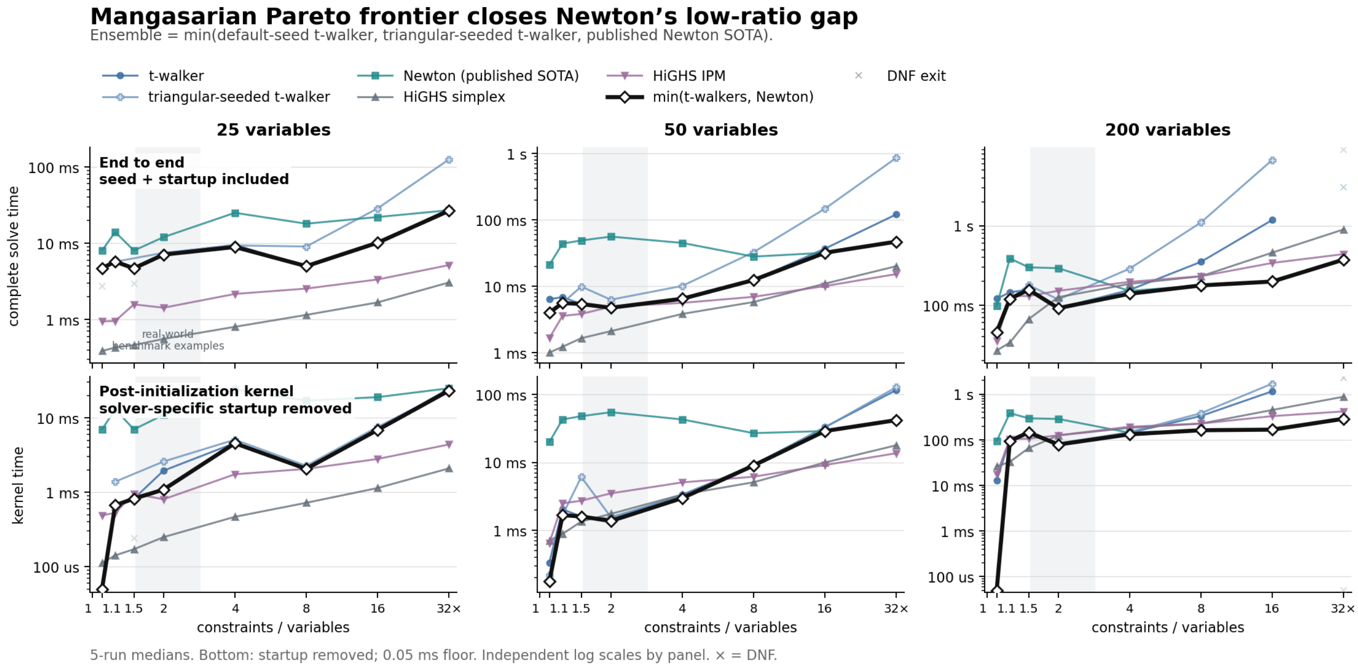

Figure 1: Complete time above and post-initialization kernel time below. The black line shows the fastest certified default-seed walker, triangular-seeded walker, or Newton result. The measured axis starts at 1.1; we retain 1.0 only as a reference. The generator changes aspect ratio and planted support geometry together, so the plot establishes a useful regime, not a theorem that aspect ratio alone is causal.

4 Accuracy, coverage, and what remains

Solver status strings do not measure accuracy on a common scale. We therefore reevaluated every available primal-dual pair on the original inequality form with one componentwise backward-error certificate. The score is the largest of four normalized errors: primal infeasibility, dual equality, violation of $y \geq 0$, and the primal-dual gap. The acceptance tolerance is 10^{-7}. DNF and resource-limit cases count as coverage failures, not zero-error samples.

For t-walker and Newton, we send each solver's dual point to the same warm-started HiGHS simplex postprocessor and then apply the same original-data certificate. We do not pass t-walker's retained basis. This procedure gives us a portable common crossover, not the proposed native basis route.

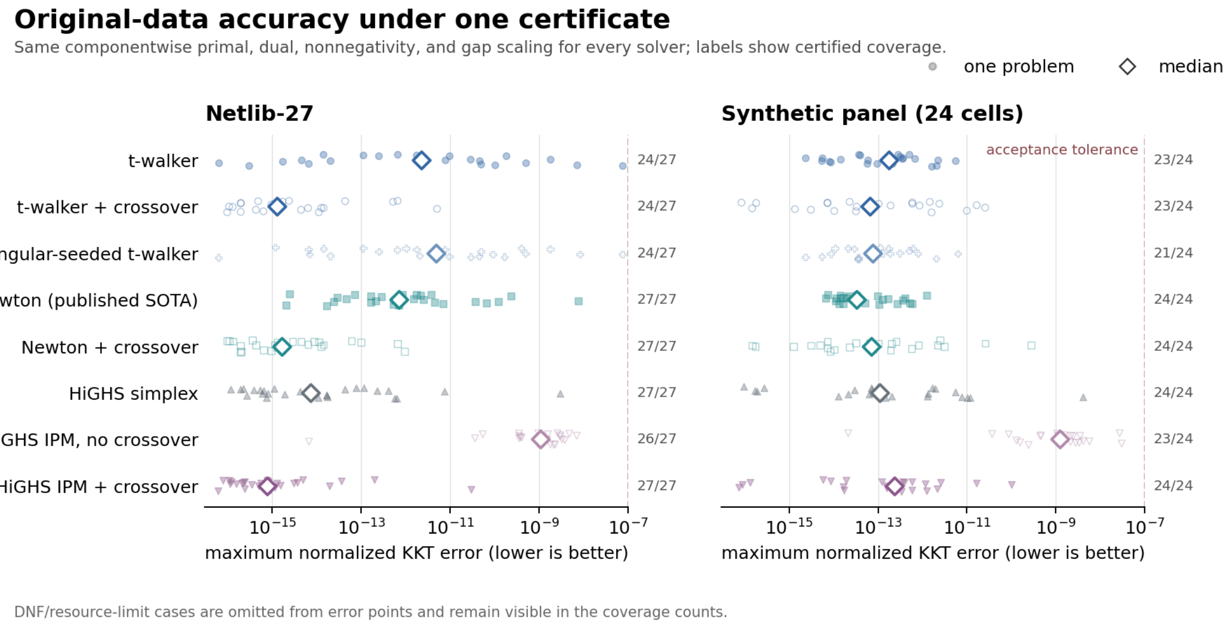

Figure 2: Each dot represents one problem under the same original-data certificate; diamonds mark medians, and the labels at right show coverage. Common crossover cuts the Netlib median error by about three orders for both methods. The synthetic results are mixed: t-walker's median improves but its tail widens, while Newton's median and tail worsen but remain inside tolerance. HiGHS IPM crossover restores one failed certificate in each suite. Crossover cannot repair a DNF because a DNF supplies no terminal point.

These results do not show that a new LP solver beats HiGHS. They show that Newton and the walker cover different regimes. On the controlled family, Newton has a reproducible weak band that the walker fills. On Netlib, their measured Pareto frontier saves about 2.0 seconds over Newton alone. The pair still trails mature solvers by more than an order of magnitude and does not certify every model.

Three problems remain. First, turn the hindsight frontier into a dispatcher that uses observed geometry—rank, weak singular values, and identification margin—rather than aspect ratio alone. Second, replace the remaining HiGHS face selectors with the maintained basis and bounded native repairs. Third, close the three-program coverage gap without slowing the cheap branch. The walker now covers part of the left-hand synthetic gap, and we measure the remaining benchmark deficit under one accuracy standard.

5 Related work and a provisional Pinar check

The projection path itself is classical. Mangasarian and Meyer established finite exactness for nonlinear LP perturbations in 1979, and Mangasarian later characterized least-norm LP solutions by projection [Mangasarian and Meyer, 1979, Mangasarian, 1984]. Madsen, Nielsen, and Pinar developed finite continuation methods on piecewise-linear paths [Madsen et al., 1996, Pınar, 1996]. Most directly, Pinar's 1997 algorithm follows a quadratic-penalty path with predictor steps, modified-Newton correction, retained factor updates, and iterative refinement [Pınar, 1997]. This work predates Mangasarian's 2002 Newton report, published in 2004 [Mangasarian, 2004, Kline and Fung, 2023].

After dualization and the change of parameter $T = 1/\tau$, Pinar's perturbed optimizer is exactly $P_D(Tb)$, the path used here. The distinction is algorithmic. Pinar evaluates selected penalty values with predictor and corrector steps; t-walker attempts to visit the next certified breakpoint. General parametric-QP active-set methods already supply the larger setting for piecewise-affine optimizer maps, ratio events, dependent-constraint exchanges, and factor updates [Best, 1996, Bock et al., 2010, Ferreau et al., 2014]. Bartels also used reduced-gradient basis exchanges in a penalty LP method [Bartels, 1980]. We therefore do not claim a new path or homotopy principle.

The exact algebra is short. Pinar perturbs a standard-form LP by

$$\min_{z \geq 0,\, Az = a} \; c^{\mathsf{T}} z + \frac{\epsilon}{2}\|z\|^2.$$

Setting $z = y$, $A = B^{\mathsf{T}}$, $a = d$, and $c = -b$ gives

$$\arg\min_{y \in D} \left(-b^{\mathsf{T}} y + \frac{\epsilon}{2}\|y\|^2 \right) = P_D(b/\epsilon) = y(t), \qquad t = 1/\epsilon.$$

Recent work sharpens finite-exactness thresholds and shows that regularization paths need not obey simple support-monotonicity rules [González-Sanz and Nutz, 2025, González-Sanz et al., 2025]. The supportable contribution here is narrower: a sparse direct breakpoint implementation of this classical path, original-data certification and rank-deficient-face repairs, and an empirical demonstration that face following complements a Mangasarian-style Newton solver in the measured low-ratio regime.

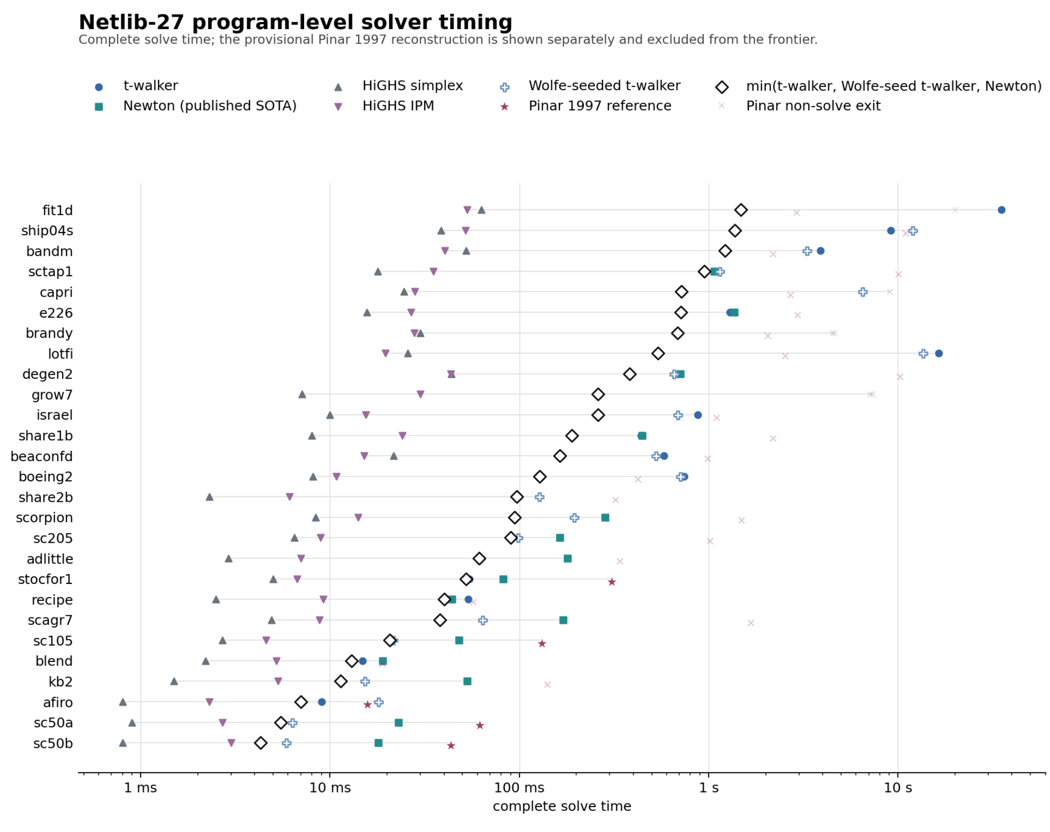

Figure 3: Complete solve time on Netlib-27. Stars mark the five certified results from the provisional Pinar reconstruction; faint crosses mark measured exits, not solves. Pinar is excluded from the black Mangasarian frontier. The other solver values are frozen cached records.

We built a correctness-first reconstruction of Pinar's algorithm and ran it on the same canonical Netlib panel with one thread and a 10-second cap. It called neither t-walker nor an LP solver. It certified `afiro`, `sc50b`, `sc50a`, `sc105`, and `stocfor1`; 18 models stopped at a numerical gate, three reached the time limit, and `grow7` was not measured because its canonical fixture was missing. The five complete Python/SciPy times ranged from 15.7 to 306.9 ms. Two identical repetitions returned the same status on every model.

These timings are directional, not a reproduction of Pinar's Fortran costs. For smaller models, our code rebuilds a cold rank-revealing SVD at every Newton and path solve; larger models use sparse LSMR. Pinar maintained and updated AAFAC factors, and his paper reports successful runs on its ten-model panel. One possible reading of our failures is that a geometric predictor and corrector may still need an explicit combinatorial basis/status policy at degenerate breakpoints. That is a hunch to test, not a conclusion from 5 of 27.

6 Sources and protocol

The TeX source and BibTeX file are the canonical PDF inputs. The Markdown note remains the working source for the planned HTML companion. Current reproductions are:

- timings: `experiments/render_solver_timing_charts.py`;
- accuracy: `experiments/bench_common_accuracy.py` and `experiments/render_common_accuracy.py`;
- provisional Pinar panel: `experiments/pinar1997/run_netlib_panel.py` and `render_provisional_netlib.py`; and
- this PDF: `paper/build_twalker_progress_note.sh`.

One thread; HiGHS presolve off; IPM uses both crossover settings. We report five interleaved runs for raw synthetic values and one deterministic warm-point run per problem for crossover accuracy. The Netlib panel combines frozen records, so read its times at order-of-magnitude precision.

Jeff Kline directed the mathematical and computational work. AI systems helped draft code, run bounded experiments, audit claims, and edit this prototype. They are neither authors nor referees. Before public release, the planned repository must identify the measured executable, preserve compact raw records, reproduce every aggregate, align the HTML and PDF claims, and record corrections. The literature review was bounded rather than exhaustive; the strongest novelty language should remain provisional until an optimization specialist reviews the equation map and bibliography.

References

Olvi L. Mangasarian and Robert R. Meyer. Nonlinear perturbation of linear programs. *SIAM Journal on Control and Optimization*, 17(6): 745–752, 1979. doi: 10.1137/0317052.

Olvi L. Mangasarian. Normal solutions of linear programs. In *Mathematical Programming Study 22*, pages 206–216. Springer, 1984. doi: 10.1007/BFb0121017.

Olvi L. Mangasarian. A newton method for linear programming. *Journal of Optimization Theory and Applications*, 121(1):1–18, 2004. doi: 10.1023/B:JOTA.0000026128.34294.77. First circulated as technical report DMI 02-02 in 2002.

Qi Huangfu and J. A. Julian Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10:119–142, 2018. doi: 10.1007/s12532-017-0130-5.

Kaj Madsen, Hans Bruun Nielsen, and Mustafa C. Pınar. A new finite continuation algorithm for linear programming. *SIAM Journal on Optimization*, 6(3):600–616, 1996. doi: 10.1137/S1052623493258556.

Mustafa C. Pınar. Linear programming via a quadratic penalty function. *Mathematical Methods of Operations Research*, 44(3):345–370, 1996. doi: 10.1007/BF01193936.

Mustafa C. Pınar. Piecewise-linear pathways to the optimal solution set in linear programming. *Journal of Optimization Theory and Applications*, 93(3):619–634, 1997. doi: 10.1023/A:1022651331550. URL `https://repository.bilkent.edu.tr/items/f49199ba-5005-4867-9f87-f9a084fd5b69`.

Jeff Kline and Glenn Fung. Linear programming with nonparametric penalty programs and iterated thresholding. *Optimization Methods and Software*, 38(1):107–127, 2023. doi: 10.1080/10556788.2022.2117356.

Michael J. Best. An algorithm for the solution of the parametric quadratic programming problem. In *Applied Mathematics and Parallel Computing*, pages 57–76. Physica-Verlag, 1996. doi: 10.1007/978-3-642-99789-1_5.

Hans Georg Bock, Christian Kirches, Andreas Potschka, and Johannes P. Schlöder. Reliable solution of convex quadratic programs with parametric active set methods. Technical report, Optimization Online, 2010. URL `https://optimization-online.org/2010/11/2828/`.

Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: A parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6:327–363, 2014. doi: 10.1007/s12532-014-0071-1.

Richard H. Bartels. A penalty linear programming method using reduced-gradient basis-exchange techniques. *Linear Algebra and its Applications*, 29:17–32, 1980. doi: 10.1016/0024-3795(80)90227-X.

Alberto González-Sanz and Marcel Nutz. Quantitative convergence of quadratically regularized linear programs. *Applied Mathematics & Optimization*, 91(3), 2025. doi: 10.1007/s00245-025-10267-1.

Alberto González-Sanz, Marcel Nutz, and Andrés Riveros Valdevenito. Monotonicity in quadratically regularized linear programs. *SIAM Journal on Optimization*, 35(2):1419–1437, 2025. doi: 10.1137/24M1685171.